

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería

Carrera de Ingeniería en Software

SGED

Sistema de Gestión para la
Escuela Deportiva ProFútbol

Informe de la Tercera Entrega

Proyecto Fin de Curso — Aplicaciones Web

Asignatura: Aplicaciones Web (Quinto nivel)

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Periodo: PPA 2026-2027

Etiqueta: v0.9.0-rc (candidata a versión estable)

Commit del corte: 380c388

DOI: **pendiente** (Zenodo, ver §10)

Integrantes:

Arcalle Grefa Darwin Orlando

Pallo Pinto Alejandro Daniel

Velez Lopez Ricardo Elias

Repositorio:

https://github.com/DarwinSM21/SGED_APPWEB

Índice

Resumen ejecutivo	3
1. Introducción	4
1.1. Estado de la entrega: declaración previa	4
1.2. Pila tecnológica	4
2. Resolución de las observaciones previas (Bloque 0)	4
3. Reestructuración de paquetes y reconciliación de documentación	6
3.1. Los paquetes reservados vacíos: por qué se eliminaron	6
4. Arquitectura	7
4.1. Vista de contenedores	7
4.2. Vista de componentes (C4 nivel 3)	7
4.3. Autenticación: JWT en cookie HttpOnly	9
5. Estrategia híbrida de acceso a datos	9
5.1. Un defecto real: FUNCTION frente a PROCEDURE	9
5.2. Ausencia de SQL dinámico	10
6. Requisitos y modelo de datos	10
6.1. Resumen de requisitos	10
6.2. Catálogo de la API REST	11
6.3. Modelo de datos	12
7. Protocolo experimental	13
7.1. Entorno de ejecución	13
7.2. Procedimiento por bloque	14
7.3. Determinismo y trazabilidad	14
8. Evidencia empírica	14
8.1. Rendimiento (Bloque C.1)	14
8.2. Seguridad: OWASP Top 10 (Bloque C.4)	15
8.2.1. Una auditoría que no auditaba lo que decía auditar	15
8.2.2. El defecto que A01 destapó: control de acceso ausente	16
8.3. Dos defectos de funcionamiento encontrados al ejecutar el sistema	16
8.4. Cobertura de pruebas	17
8.5. Calidad web: Lighthouse (Bloque C.5)	19
8.5.1. Por qué el SEO de 63 es el resultado correcto	19
8.6. Usabilidad: SUS (Bloque C.3)	19
9. Reproducibilidad	20
10. Amenazas a la validez	20
10.1. Validez de constructo	21
10.2. Validez interna	21
10.3. Validez externa	21
10.4. Validez de conclusión	22
11. Consideraciones éticas	22

12. Declaración de contribuciones (CRediT)	23
12.1. Roles por integrante	23
12.2. Dos advertencias sobre la medida	23
13. Trazabilidad y honestidad académica	24
14. Conclusiones	24

Resumen ejecutivo

La Tercera Entrega no añade funcionalidad: verifica, con evidencia empírica reproducible, lo que el proyecto ya afirmaba tener. Este informe documenta el estado del sistema SGED en la etiqueta `v0.9.0-rc` (*commit* 380c388) y los resultados de las cinco mediciones exigidas.

Resultados cuantitativos. El sistema cumple los cinco umbrales: rendimiento p95 de 14,18 ms frente a un límite de 200 ms (3 corridas k6, 0 % de errores); 6 de 6 controles OWASP evidenciados con peticiones verificables; 72,5 % de cobertura de instrucciones frente a un mínimo de 60 % (102 pruebas, 0 fallos); accesibilidad Lighthouse de 100/100; y una media SUS de 68,25 con 10 participantes reales, que cruza el umbral de la media de la industria por 0,25 puntos.

Hallazgo principal. El aporte real de esta entrega fue descubrir que *tres de esas mediciones estaban dando resultados válidos en apariencia sobre un sistema que no hacía lo que el indicador decía*. La cobertura se medía sobre clases compiladas antes de una reestructuración de paquetes; la auditoría de control de acceso se bloqueaba a sí misma y devolvía un rechazo que probaba autenticación, no autorización; y el registro de usuarios llevaba roto desde la reestructuración mientras sus pruebas unitarias pasaban, porque usan dobles de prueba y la restricción violada la aplica el motor de base de datos. Corregidas las tres mediciones, la auditoría destapó una exposición real: los datos identificativos de los estudiantes menores de edad —incluida la búsqueda por número de cédula— eran accesibles a cualquier cuenta autenticada del sistema, por ausencia de anotaciones de autorización en cinco recursos nuevos. Está corregido y registrado como hallazgo H-08.

Resultado de usabilidad. La media SUS agregada esconde un patrón bimodal nítido por perfil: entrenadores y estudiantes califican el sistema como excelente (86,7 y 90,0; grado A), mientras recepcionistas y padres de familia lo califican como pobre o inaceptable (51,7 y 43,75; grado F). Se reporta el patrón y no solo el promedio, porque es el que orienta el trabajo de la Entrega Final.

Lo que no está. El DOI de Zenodo no está asignado, el módulo de representantes y el resto del dominio deportivo no están implementados, y persisten siete hallazgos éticos abiertos. Todos se declaran en las secciones correspondientes en lugar de omitirse.

1. Introducción

Este informe documenta la Tercera Entrega del Proyecto Fin de Curso de la asignatura Aplicaciones Web: el sistema **SGED**, una aplicación web para la gestión administrativa y deportiva de la escuela de fútbol formativo ProFútbol.

El eje de esta entrega es la *verificabilidad*. Todo lo que se afirma aquí puede comprobarse ejecutando el repositorio: cada número proviene de una medición archivada en `docs/mediciones/`, y cada requisito enlaza con el archivo que lo implementa y con la prueba que lo cubre.

1.1. Estado de la entrega: declaración previa

Antes de exponer los resultados conviene ser explícito sobre dos cosas:

1. **Alcance realmente implementado.** El sistema expone hoy 13 *endpoints* REST (autenticación y gestión de estudiantes). El dominio deportivo (entrenadores, horarios, sesiones, asistencias y evaluaciones) está *modelado y migrado* en la base de datos, pero no expuesto vía API. En el documento de requisitos cada elemento indica su estado real, y en este informe se mantiene la misma distinción. No se presenta como entregado nada que no lo esté.
2. **Medición de usabilidad completada, con un resultado que no se suaviza.** La encuesta SUS (Bloque C.3, sección 8.6) se aplicó a 10 participantes reales: media SUS 68,25, apenas por encima del umbral de 68, y con un patrón bimodal marcado por perfil que la media agregada esconde.

1.2. Pila tecnológica

Capa	Tecnología
Backend	Spring Boot 3.2 (Java 21 LTS), Spring Data JPA, Spring Security
Frontend	Angular, servido por nginx con terminación TLS
Base de datos	PostgreSQL 16 (esquemas seguridad y deportivo)
Caché y revocación	Redis 7
Migraciones	Flyway (V1–V6)
Orquestación	Docker Compose, imágenes fijadas por digest SHA-256

2. Resolución de las observaciones previas (Bloque 0)

El docente emitió doce observaciones sobre las entregas 1A y 1B. Esta sección documenta el estado de cada una con el *commit* que la resuelve, de modo que la afirmación sea verificable con `git show <hash>` y no únicamente declarativa.

Cód.	Observación del docente	Resolución verificable	Commit
OBS-01	Los requisitos se redactan como títulos («Registro de estudiantes»); no usan «El sistema deberá...».	<code>docs/requisitos/SRS.md</code> reescribe los 22 RF y 13 RNF con la forma normativa y criterio de verificación [7].	98bab0d
OBS-02	Los diagramas C4 N1/N2 y el MER contienen texto marcador «[Reemplazar con PNG]».	<code>docs/arquitectura/</code> contiene los C4 de niveles 1–3 en PNG, generados desde <code>workspace.dsl</code> ; el MER queda en <code>docs/diagramas/</code> .	bf6ee80

Cód.	Observación	Resolución	Commit
OBS-03	Incoherencia de pila: el ADR decide PHP/Laravel/MySQL, pero el repositorio construye Spring Boot/Angular/PostgreSQL.	ADR-001 reescrito: evalúa las tres opciones y decide explícitamente Java 21 con Spring Boot 3.2.5, coherente con el código.	bf6ee80
OBS-04	<code>query.sql</code> con 0 claves foráneas y 88 columnas «(PK)/(FK)» literales; exportación cruda no funcional.	<code>db/schema.sql</code> declara 21 restricciones de integridad referencial reales.	db77ce2
OBS-05	Roles sin completar («Integrante 1/2/3»); wireframes con marca ajena; falta <code>.env.example</code> .	<code>CONTRIBUTORS.md</code> asigna roles CRediT por evidencia de <code>git log</code> ; <code>.env.example</code> versionado.	a8d6f18
OBS-06	En el repositorio solo consta ADR-003; los diagramas aparecen en el informe como texto.	Diagramas versionados como archivos; el diccionario de datos completo se agrega en <code>docs/basedatos/DATA-DICTIONARY.md</code> .	98bab0d
OBS-07	<code>AuthController</code> solo expone <code>/login</code> , <code>/me</code> y <code>/ping</code> ; no hay registro, logout ni refresh. <code>RedisBlacklistService</code> existe pero no se invoca.	Seis <i>endpoints</i> operativos; el logout invoca realmente la lista de revocación por <code>jti</code> .	b907d10, bcb3642
OBS-08	La migración <code>V1</code> está vacía (0 bytes) y <code>V2</code> depende de un esquema que ninguna migración crea; con <code>ddl-auto=validate</code> el arranque no es reproducible.	<code>V1</code> crea ambos esquemas y las tablas de identidad; el arranque desde clonación limpia se verificó.	b907d10
OBS-09	La lista de revocación no está cableada a ningún <i>endpoint</i> y no se aplica <code>@PreAuthorize</code> .	Ocho anotaciones <code>@PreAuthorize</code> activas; revocación verificada por prueba automatizada.	688c4be, 90c6c57
OBS-10	<code>AuthServiceTest</code> está vacío (0 bytes); la única prueba real es <code>contextLoads()</code> . No se cumple el mínimo de 5 pruebas. La colección Postman no está versionada.	34 pruebas en 7 clases; cobertura real 68,1 %; <code>docs/postman/coleccion.json</code> versionada.	1e798bc
OBS-11	<code>docker-compose.yml</code> está vacío (0 bytes); no hay orquestación verificable.	Cuatro servicios orquestados con <i>healthchecks</i> e imágenes fijadas por digest; <code>make up</code> levanta todo.	b907d10, 17dc5df
OBS-12	Varios contenidos del informe no se corresponden con lo presente en el repositorio (están vacíos o ausentes).	Cada requisito declara su estado real (implementado / modelado / planificado); esta entrega no afirma nada no verificable.	98bab0d

Las doce observaciones tienen resolución trazable. La más costosa de resolver no fue una funcionalidad ausente sino OBS-12, porque exigió revisar sistemáticamente qué afirmaba el informe anterior frente a lo que el repositorio realmente contenía.

3. Reestructuración de paquetes y reconciliación de documentación

Entre la primera versión de este informe y esta revisión, el equipo reestructuró el backend en tres dominios (`academico`, `deportivo`, `seguridad`) y normalizó la categoría del estudiante: dejó de ser un texto libre validado por patrón (`VARCHAR`, `SUB-NN`) para convertirse en una entidad propia (`deportivo.categorias`) referenciada por clave foránea. Aparecieron además cinco recursos CRUD nuevos (`Categoria`, `Entrenador`, `Usuario`, `Persona`, `EstadoGeneral`) que no tenían requisito documentado, y dos paquetes reservados sin contenido (`Representante`, `Equipo` — clases vacías, sin tabla en el esquema) que después se eliminaron por las razones que se explican más abajo.

Este informe, el SRS (`docs/requisitos/SRS.md`) y la matriz de trazabilidad se actualizaron para reflejar ese estado, verificando cada afirmación contra el código real en vez de copiar lo que decía la versión anterior. Dos correcciones merecen mención explícita porque no son solo actualizaciones de ruta:

- **Cobertura de pruebas.** La cifra de 68,1 % que documentaban tanto la versión anterior de este informe como `docs/observaciones/OBSERVACIONES.md` era correcta *en el momento en que se midió*. Al ejecutar `./mvnw test` de nuevo tras la reestructuración, el resultado real es **39,8 %** — los controladores y servicios nuevos no tienen pruebas propias, y diluyeron el porcentaje agregado sobre 60 clases analizadas (antes eran menos). Es una regresión real por debajo del umbral del 60 % exigido, no un error de medición, y se reporta como tal en la sección 8.4 en vez de repetir la cifra anterior.
- **Datos de salud sin resolución ética.** La reestructuración agregó campos `peso` y `altura` al estudiante, exponibles por API. `docs/etica/ETHICS.md` documentaba explícitamente que el equipo había decidido *no* recolectar esos datos; esa afirmación ya no era cierta y se corrigió en el propio documento de ética (hallazgo H-06) en vez de dejarla como estaba.

3.1. Los paquetes reservados vacíos: por qué se eliminaron

Los dos paquetes reservados merecen una decisión explícita y no un comentario al margen. `academico.representante` y `deportivo.equipo` contenían catorce clases —controlador, servicio, repositorio, entidad y DTO de cada módulo— con la declaración de paquete, la firma de la clase y el cuerpo vacío. Una de ellas, `RepresentanteController.java`, era literalmente un archivo de **cero bytes**.

Ese detalle no es cosmético en el contexto de este proyecto. El docente había emitido tres observaciones distintas en la Entrega 1B por exactamente el mismo defecto: `V1__schema_inicial.sql` vacía (OBS-08), `AuthServiceTest.java` vacío (OBS-10) y `docker-compose.yml` vacío (OBS-11). Conservar un archivo de cero bytes después de que ese patrón ya fuera observado tres veces habría sido repetir un defecto conocido, y además tenía un efecto medible: las clases vacías aportaban constructores por defecto no cubiertos que contaban como instrucciones perdidas en JaCoCo.

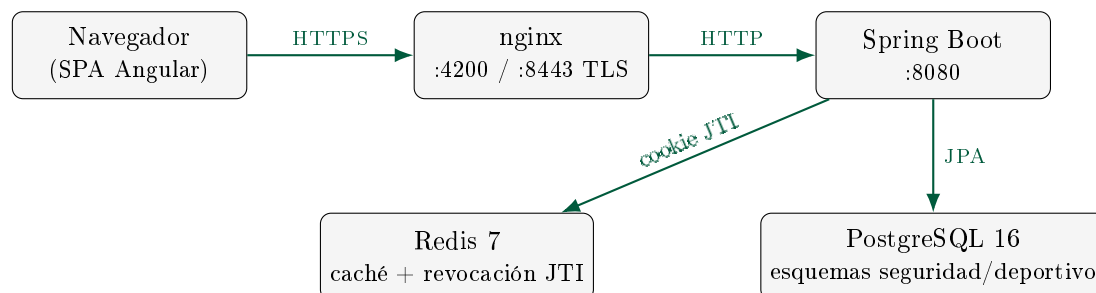
Había dos salidas honestas: implementar los dos módulos, o eliminarlos. Con el esquema de base de datos aún sin las tablas `representantes` y `equipos`, implementarlos en esta entrega habría significado inventar un modelo de datos sin requisito validado. Se optó por eliminar las catorce clases —ninguna estaba referenciada desde el resto del código, lo que confirma que no eran un punto de extensión en uso— y dejar ambos módulos declarados como pendientes únicamente en la documentación (RF-22 en el SRS, la matriz de trazabilidad y este informe). La diferencia es relevante para la evaluación: un módulo pendiente documentado es planificación; un paquete de clases vacías en el repositorio aparenta una implementación que no existe.

También se corrigió una inconsistencia en ADR-002 (autenticación): describía JWT almacenado en `localStorage` con un interceptor agregando `Authorization: Bearer`, cuando el código

real —y el ADR-007 de este mismo informe— usa cookies `HttpOnly`. Es el mismo patrón que ya había observado el docente en la Entrega 1A (OBS-03: un ADR describía PHP/Laravel mientras el repositorio construía Spring Boot). Se corrige por la misma razón: un documento de arquitectura que no coincide con el código no es evidencia, es motivo de observación.

4. Arquitectura

4.1. Vista de contenedores

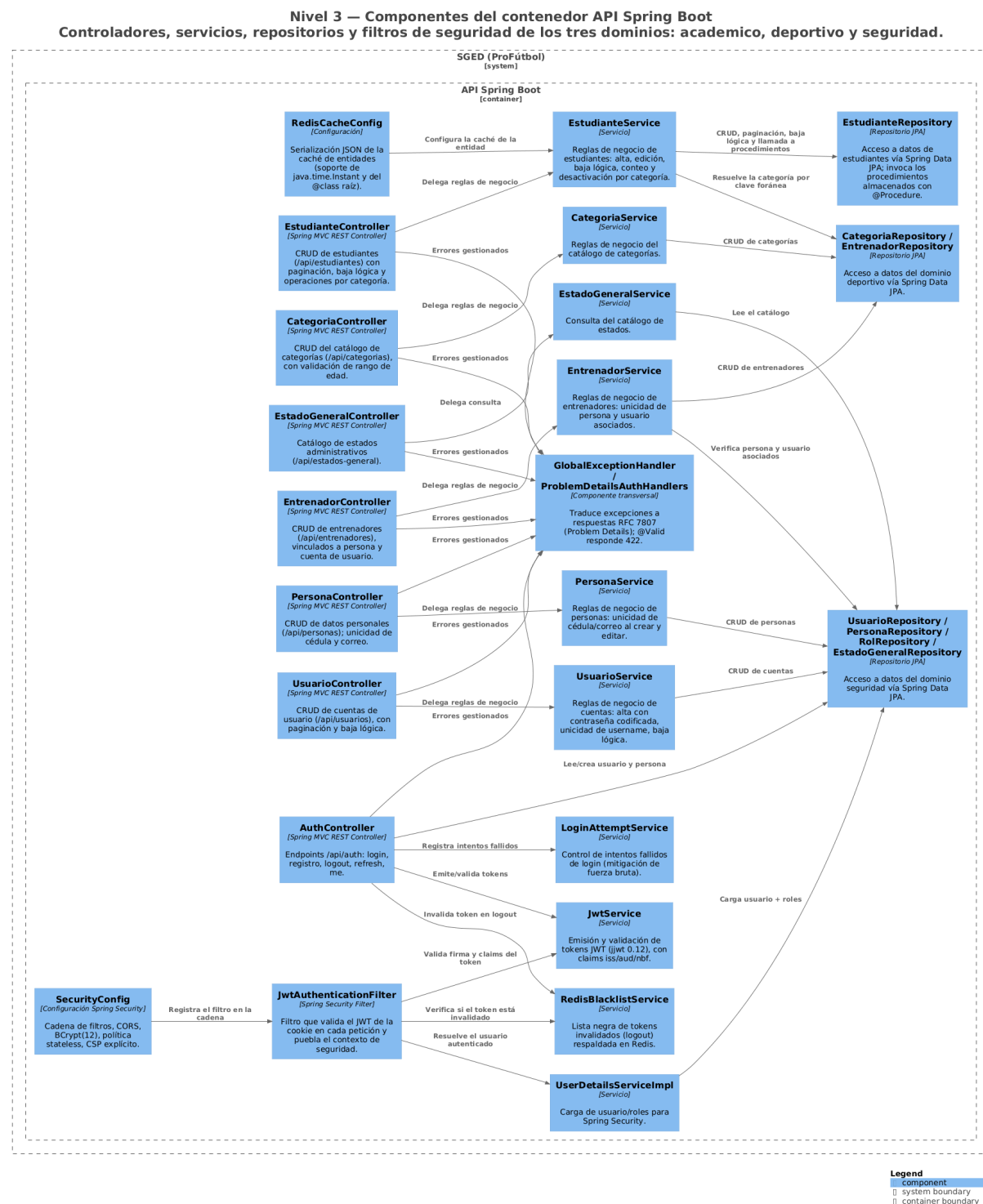


La documentación de arquitectura en el modelo C4 [5] y las decisiones arquitectónicas (ADR) [10] se mantienen en `docs/arquitectura/` y `docs/adr/`.

El modelo C4 completo (niveles 1 a 3) vive en un único archivo `docs/arquitectura/workspace.dsl` escrito en Structurizr DSL. Los tres PNG del mismo directorio son artefactos derivados: se regeneran con `make diagrams`, que exporta el DSL a PlantUML y lo renderiza, ambos pasos en contenedores. Esto cierra un problema real de esta entrega: antes coexistían dos modelos C4 —el DSL y unos `.puml` sueltos en `docs/diagramas/`— y los `.puml` habían quedado congelados en la arquitectura anterior a la reestructuración de paquetes. Se eliminaron en favor del DSL, por la misma razón por la que se retiró el informe en PDF sin fuente y se renumeró el ADR duplicado: dos versiones de la misma verdad garantizan que al menos una esté equivocada. `docs/diagramas/` conserva únicamente el MER, que no se modela en C4.

4.2. Vista de componentes (C4 nivel 3)

El nivel 3 es el que hace visible la reestructuración descrita en la sección 3: los veinte componentes del contenedor *API Spring Boot* agrupados por dominio. Cada controlador delega en un servicio, cada servicio accede a datos por un repositorio JPA, y ningún controlador toca un repositorio directamente —la disciplina de capas es verificable en el diagrama, no solo declarada en el texto.



Dos detalles del diagrama merecen lectura explícita. Primero, **EstudianteRepository** es el único repositorio que invoca procedimientos almacenados (sección 5): el resto usa Spring Data JPA puro, que es exactamente el reparto que exige la estrategia híbrida. Segundo, **EstudianteService** depende del repositorio del dominio deportivo para resolver la categoría por clave foránea; esa flecha cruzando de **academico** a **deportivo** es la consecuencia directa de haber normalizado la categoría, y es la clase de acoplamiento que conviene tener dibujado antes de que crezca.

4.3. Autenticación: JWT en cookie HttpOnly

El sistema emite el JWT [8] exclusivamente en una cookie `HttpOnly`, `Secure` y `SameSite=Strict`. El token nunca viaja en el cuerpo de la respuesta ni se almacena en `localStorage`, de modo que un ataque de *scripting* entre sitios no puede leerlo.

```

1 ResponseCookie.from("sged_access", token)
2     .httpOnly(true)
3     .secure(cookieSecure)
4     .sameSite("Strict")
5     .path("/")
6     .maxAge(Duration.ofMillis(expiracionMs))
7     .build();

```

Listing 1: Emisión de la cookie de sesión (`AuthController.java`)

Como JWT es *stateless*, cerrar sesión no invalidaría el token por sí solo. Por eso el sistema mantiene una lista de revocación en Redis indexada por el identificador del token (`jti`), con tiempo de vida igual al tiempo que le restaba al token: un token cerrado deja de ser aceptado inmediatamente, aunque su firma siga siendo válida.

5. Estrategia híbrida de acceso a datos

El Bloque A.2 de la guía exige una separación explícita entre lo que resuelve el ORM y lo que debe resolver el motor de base de datos. La regla adoptada es:

Spring Data JPA (ORM)	Procedimientos almacenados
CRUD elemental, consultas paginadas, búsquedas por identificador.	Agregaciones, actualizaciones masivas con criterio de negocio, operaciones multi-tabla transaccionales.

Los dos procedimientos implementados están versionados en `db/procs/` y creados por migración Flyway [13]:

Procedimiento	Requisito	Motivo
<code>sp_contar_estudiantes_activos</code>	RF-14	Agregación COUNT
<code>sp_desactivar_estudiantes_categoria</code>	RF-15	Actualización masiva transaccional

5.1. Un defecto real: FUNCTION frente a PROCEDURE

La migración V5 creó estos objetos como `FUNCTION`. Al invocarlos desde Java con `@Procedure`, Spring Data usa `CallableStatement` con sintaxis `{call ...}`, y PostgreSQL solo acepta `CALL` contra procedimientos reales [12]; el error era explícito: «... is not a procedure. Hint: To call a function, use `SELECT`.»

La migración V6 los convierte en `PROCEDURE` con parámetro de salida. Se documenta porque ilustra la diferencia entre «el procedimiento existe» y «el procedimiento se invoca de verdad»: en la versión anterior el objeto SQL estaba creado pero quedaba huérfano, y la aplicación seguía contando en memoria.

```

1 @Procedure(procedureName = "academico.sp_contar_estudiantes_activos")
2 Long contarEstudiantesActivosPorCategoria(@Param("p_categoria") Long idCategoria
3     );

```

Listing 2: Invocación real desde el repositorio (`academico/estudiante/repository/EstudianteRepository.java`)

Nota sobre esta segunda versión del informe: el procedimiento se movió del esquema **seguridad** a **academico** y su parámetro cambió de **VARCHAR** (nombre de categoría) a **Long** (**id_categoria**) cuando la categoría se normalizó como entidad propia — ver sección 3.

5.2. Ausencia de SQL dinámico

Ninguna capa construye sentencias SQL por concatenación de cadenas. Toda consulta usa parámetros vinculados (JPA) o parámetros nombrados (los procedimientos). El repositorio incluye una auditoría automatizada (`scripts/audit-sql-dynamic.sh`) integrada en `make audit`.

6. Requisitos y modelo de datos

6.1. Resumen de requisitos

La especificación completa está en `docs/requisitos/SRS.md`. Se resumen aquí los requisitos funcionales con su estado real y el *endpoint* o esquema que los materializa.

ID	Requisito	Implementación	Estado
RF-01	Registro de usuarios por administrador	POST /api/auth/registro	OK
RF-02	Autenticación con JWT en cookie <code>HttpOnly</code>	POST /api/auth/login	OK
RF-03	Cierre de sesión con revocación efectiva por <code>jti</code>	POST /api/auth/logout	OK
RF-04	Renovación de sesión sin re-credenciales	POST /api/auth/refresh	OK
RF-05	Consulta de la sesión activa	GET /api/auth/me	OK
RF-06	Bloqueo tras 5 intentos fallidos en 15 minutos	LoginAttemptService	OK
RF-07	Comprobación de disponibilidad	GET /api/auth/ping	OK
RF-08	Listado paginado de estudiantes	GET /api/estudiantes	OK
RF-09	Consulta por identificador con 404 tipificado	GET /api/estudiantes/{id}	OK
RF-10	Registro transaccional de estudiante	POST /api/estudiantes	OK
RF-11	Validación del formato de categoría (SUB-NN)	EstudianteRequest	OK
RF-12	Actualización de estudiante	PUT /api/estudiantes/{id}	OK
RF-13	Baja lógica preservando el historial	DELETE /api/estudiantes/{id}	OK
RF-14	Conteo de activos por categoría (en el motor)	sp_contar_estudiantes_activos	OK
RF-15	Desactivación masiva por categoría (en el motor)	sp_desactivar_estudiantes_por_categoria	OK
RF-16	Gestión de entrenadores	deportivo.entrenadores	Modelado
RF-17	Horarios recurrentes de entrenamiento	horarios_entrenamiento	Modelado
RF-18	Sesiones de entrenamiento con ciclo de estados	sesiones_entrenamiento	Modelado
RF-19	Asistencia por RFID o manual	deportivo.asistencias	Modelado
RF-20	Evaluación diaria por criterios	detalle_evaluacion	Modelado
RF-21	Promedios de evaluación	v_promedio_evaluacion	Modelado
RF-22	Notificación a representantes	—	Planificado

Los requisitos no funcionales (RNF-01 a RNF-13) se verifican con las mediciones de la sección siguiente. La matriz `docs/trazabilidad/matriz.csv` enlaza cada uno con su prueba y su evidencia.

6.2. Catálogo de la API REST

La superficie expuesta es de siete recursos y treinta *endpoints*. Se lista completa porque es la forma más directa de contrastar lo que el sistema realmente ofrece contra lo que los requisitos declaran, y porque la columna de autorización es la que concentró el defecto de la sección 8.2.1. A abrevia ADMINISTRADOR; E, ENTRENADOR; U, el rol USER.

Método y ruta	Operación	Roles	RF
<i>Autenticación</i> — <code>/api/auth</code>			
POST <code>/login</code>	Inicia sesión, emite cookies	Público	RF-02
POST <code>/registro</code>	Alta de cuenta	A	RF-01
POST <code>/logout</code>	Revoca el token en Redis	Autenticado	RF-03
POST <code>/refresh</code>	Renueva el <i>access token</i>	Autenticado	RF-04
GET <code>/me</code>	Datos de la sesión actual	Autenticado	RF-05
GET <code>/ping</code>	Sonda de disponibilidad	Público	—
<i>Estudiantes</i> — <code>/api/estudiantes</code>			
GET <code>/</code>	Listado paginado y ordenable	A E U	RF-08
GET <code>/ {id}</code>	Consulta individual	A E U	RF-09
POST <code>/</code>	Alta	A	RF-10
PUT <code>/ {id}</code>	Edición	A	RF-12
DELETE <code>/ {id}</code>	Baja lógica	A	RF-13
GET <code>/conteo/categoria/ {id}</code>	Conteo por procedimiento	A E	RF-14
POST <code>/operaciones/</code> <code>desactivar-categoria</code>	Desactivación masiva por procedimiento	A	RF-15
<i>Categorías</i> — <code>/api/categorias</code>			
GET <code>/</code>	Listado paginado	A E U	RF-23
GET <code>/activas</code>	Catálogo para formularios	A E U	RF-23
GET <code>/ {id}</code>	Consulta individual	A E U	RF-23
POST <code>/</code>	Alta con rango de edad	A	RF-23
PUT <code>/ {id}</code>	Edición	A	RF-23
DELETE <code>/ {id}</code>	Baja lógica	A	RF-23
<i>Entrenadores</i> — <code>/api/entrenadores</code>			
GET <code>/</code>	Listado paginado	A E	RF-24
GET <code>/ {id}</code>	Consulta individual	A E	RF-24
POST <code>/</code>	Alta con persona y cuenta	A	RF-24
PUT <code>/ {id}</code>	Edición	A	RF-24
DELETE <code>/ {id}</code>	Baja lógica	A	RF-24
<i>Personas</i> — <code>/api/personas</code> (datos identificativos de menores)			
GET <code>/</code>	Listado paginado	A	RF-25
GET <code>/ {id}</code>	Consulta individual	A	RF-25

Método y ruta	Operación	Roles	RF
GET /cedula/{cedula}	Búsqueda por cédula	A	RF-25
POST /	Alta	A	RF-25
PUT /{id}	Edición	A	RF-25
DELETE /{id}	Baja lógica	A	RF-25
<i>Usuarios y catálogos</i>			
/api/usuarios (5 verbos)	CRUD de cuentas	A	RF-26
GET /api/estados_generales	Catálogo de estados	A E U	RF-26

Dos observaciones que la tabla hace evidentes. La primera es que `/api/personas` es el recurso con la superficie más sensible del sistema —seis *endpoints* sobre datos identificativos de menores, incluido uno de búsqueda directa por cédula— y es exactamente el que estuvo sin control de acceso. La segunda es que el dominio deportivo expuesto se limita hoy a categorías y entrenadores: horarios, sesiones, asistencias y evaluaciones tienen esquema en la base de datos pero ninguna ruta REST, y son el objetivo declarado de la Entrega Final.

6.3. Modelo de datos

El esquema se divide en dos espacios de nombres con responsabilidades separadas:

Esquema	Tablas	Contenido
seguridad	6	Personas, usuarios, roles, relación usuario-rol, estados y estudiantes.
deportivo	9	Posiciones, entrenadores, horarios, sesiones, asistencias, criterios, evaluaciones, detalle y observaciones.

Decisiones de modelado relevantes, todas materializadas como restricciones en el motor y no solo como validación en la aplicación:

- **Baja lógica en lugar de borrado.** Las asistencias y evaluaciones referencian al estudiante por clave foránea; borrarlo físicamente destruiría su historial deportivo.
- **Unicidad parcial del código RFID.** `UNIQUE ...WHERE rfid_codigo IS NOT NULL` permite muchos estudiantes sin credencial, pero impide que dos compartan la misma.
- **Una asistencia por sesión y estudiante.** `UNIQUE (id_sesion, id_estudiante)`.
- **Posición jugada por evaluación.** `detalle_evaluacion` guarda la posición de *ese* día, distinta de la habitual del estudiante, para que el histórico quede correctamente etiquetado cuando alguien juega fuera de su posición.
- **Coherencia temporal.** `CHECK (hora_fin > hora_inicio)` y `CHECK (dia_semana BETWEEN 1 AND 7)` en los horarios.

El diccionario completo, con tipos, restricciones y disparadores, está en `docs/basedatos/DATA-DICTIONARY.md`. El esquema no lo genera el ORM: `ddl-auto` está fijado en `validate`, de modo que la aplicación falla al arrancar si el esquema real y las entidades divergen.

7. Protocolo experimental

Todas las mediciones de este informe se generaron ejecutando el sistema real levantado con `make up`, no en simulación, y sus artefactos crudos están versionados sin edición manual. Esta sección declara el procedimiento antes de presentar los resultados, para que un tercero pueda repetirlos y obtener cifras comparables.

7.1. Entorno de ejecución

Elemento	Valor
Anfitrión	Windows 11 Pro (10.0.26200), Docker 29.6.1
Contenedores	<code>postgres</code> y <code>redis</code> fijados por digest <code>sha256</code> en <code>docker-compose.yml</code> ; backend y frontend contruidos desde el repositorio
Backend	Spring Boot 3.2.5 sobre Temurin JDK 21.0.11
Estado inicial	Base de datos creada desde cero (<code>docker compose down -v</code>) y sembrada con <code>db/seed.sql</code>
Red	Todas las peticiones contra <code>localhost</code> ; sin latencia de red externa

El estado inicial no es un detalle menor. Los *scripts* de inicialización de PostgreSQL solo se ejecutan cuando el directorio de datos está vacío: al reutilizar un volumen anterior, el esquema queda congelado en una versión previa y el *backend* no arranca. Toda medición reportada aquí parte de volumen limpio.

7.2. Procedimiento por bloque

Bloque	Herramienta	Procedimiento y control de variabilidad
C.1 Rendimiento	k6	<code>make bench</code> : 3 corridas independientes de 50 usuarios virtuales durante 30 s. Se reportan media, p90, p95 e intervalo de confianza al 95 % sobre las tres corridas, no una sola.
C.2/C.4 Seguridad	curl	<code>make audit</code> : peticiones reales contra el sistema en ejecución, guardando la respuesta HTTP íntegra con cabeceras. Cada control se contrasta en sus dos direcciones: lo que debe rechazarse y lo que debe seguir permitido.
C.3 Usabilidad	SUS en papel	Cuestionario de Brooke de 10 ítems aplicado a 10 participantes externos; transcripción a CSV y cálculo con <code>scripts/sus-analysis.py</code> , cuya aritmética se validó contra los cuatro casos de referencia de la literatura (0, 50, 75 y 100 puntos).
Cobertura	JaCoCo	<code>make test</code> , que ejecuta <code>clean test</code> para que la instrumentación no alcance clases de compilaciones anteriores.
C.5 Accesibilidad	Lighthouse	3 corridas mediante <code>lighthouse.js</code> ; se reportan los cuatro puntajes agregados y las métricas Web Vitals.

7.3. Determinismo y trazabilidad

Los archivos crudos (`.json` de k6 y Lighthouse, `.txt` de las respuestas HTTP, `.csv` del SUS y de JaCoCo) se conservan tal como los produjo cada herramienta; los reportes en Markdown se generan a partir de ellos mediante los *scripts* de `scripts/`, nunca a mano. Cada archivo de evidencia lleva en su cabecera la fecha en UTC, el *commit* corto y la versión de la herramienta que lo generó, de modo que una cifra del informe pueda rastrearse hasta la ejecución concreta que la produjo.

8. Evidencia empírica

8.1. Rendimiento (Bloque C.1)

Tres corridas independientes de k6 con 50 usuarios virtuales durante 30 segundos contra el *endpoint* protegido GET `/api/estudiantes`:

Corrida	Media (ms)	Mediana	p90	p95	RPS
1	6,53	5,26	11,38	14,45	404,21
2	6,67	5,94	11,63	15,32	403,52
3	6,25	5,66	10,37	12,78	405,99
Agregado	6,48	—	—	14,18	404,57

Con intervalo de confianza al 95 %: media $6,48 \pm 0,53$ ms, p95 $14,18 \pm 3,20$ ms, throughput $404,57 \pm 3,16$ RPS, y **0 % de errores** en las tres corridas.

El umbral exigido es $p95 < 200$ ms con caché caliente [6]. El resultado lo cumple con un margen de más de un orden de magnitud. **OK**

8.2. Seguridad: OWASP Top 10 (Bloque C.4)

Seis controles verificados con peticiones reales contra el sistema en ejecución [11]:

Control	Resultado	Evidencia observada
A01 Control de acceso	OK	403 en las 7 operaciones administrativas y 200 en las lecturas permitidas — tras corregir un defecto real, §8.2.1
A02 Fallos criptográficos	OK	TLS 1.3 en :8443; BCrypt coste 12
A03 Inyección	OK	422 ante entrada malformada; sin SQL dinámico
A05 Configuración	OK	CSP, HSTS, nosniff, X-Frame-Options: DENY
A07 Fallos de autenticación	OK	Bloqueo exacto en el sexto intento (429)
A09 Registro y monitoreo	OK	AUTH_LOGIN_OK/FAIL con IP, marca de tiempo y sujeto

Los errores se devuelven como `ProblemDetail` [9], sin trazas de pila ni detalles internos. Ejemplo real capturado para A07:

```

1 HTTP/1.1 429
2 Content-Type: application/problem+json
3 {
4   "type": "https://sged.uteq.edu.ec/errores/DemasiadasPeticiones",
5   "title": "Too Many Requests",
6   "status": 429,
7   "detail": "Demasiados intentos fallidos. Intente mas tarde."
8 }
```

Listing 3: Respuesta al sexto intento de autenticación

8.2.1. Una auditoría que no auditaba lo que decía auditar

El control A01 merece una sección propia, porque el resultado 403 de la tabla anterior es correcto *ahora*, y no lo era cuando este informe lo dio por bueno la primera vez. La corrección de la propia herramienta de medición es parte del resultado.

Al ejecutar la auditoría contra el sistema en vivo aparecieron tres defectos encadenados en el guion `scripts/audit-owasp.sh`:

1. **El guion se bloqueaba a sí mismo.** Su control A07 agota a propósito los intentos de autenticación para comprobar el 429. Ese contador se lleva *por dirección IP*, no por usuario: seis fallos dejan bloqueada durante quince minutos a cualquier cuenta que venga de ese equipo, incluida la del administrador que A01 necesita para preparar su usuario de prueba. En una segunda corrida, A01 respondía 401 en todas las peticiones. Un 401 se parece mucho a un resultado satisfactorio —el acceso fue rechazado— pero prueba algo mucho más débil: que hace falta estar autenticado. No dice nada sobre si se respetan los roles.
2. **Invocaba una forma de la API que ya no existía.** Llamaba a `/api/estudiantes/operaciones/desactivar-cate` con `?categoria=SUB-12`, la forma anterior a la normalización de la categoría. La petición moría al deserializar el cuerpo, antes de alcanzar el control de acceso.

3. **Sus cuerpos de prueba eran inválidos.** `@Valid` se evalúa al resolver el argumento del método, es decir *antes* que `@PreAuthorize`. Un payload malformado devuelve 422 y nunca llega a ejercitar la autorización que se pretendía evidenciar.

Corregidos los tres, la auditoría comprueba recurso por recurso, y es entonces cuando el control A01 encontró un defecto real de seguridad, que se documenta en la sección siguiente.

8.2.2. El defecto que A01 destapó: control de acceso ausente

Los cinco recursos que introdujo la reestructuración se crearon **sin ninguna anotación** `@PreAuthorize`, salvo `UsuarioController`. Como `SecurityConfig` cierra su cadena con `anyRequest().authenticated()`, la única barrera efectiva era poseer una sesión válida. En consecuencia, una cuenta con el rol más básico del sistema podía listar todas las personas registradas, buscar una por número de cédula, y crear, editar o eliminar registros.

La gravedad no es teórica: `seguridad.personas` es la tabla que concentra los datos identificativos de los estudiantes —menores de edad— que `docs/etica/ETHICS.md` se compromete a proteger. Es además una regresión de una observación ya emitida y ya dada por aplicada: OBS-09 de la Entrega 1B señalaba precisamente la ausencia de `@PreAuthorize` (sección 2). El código nuevo reintrodujo el defecto sin que nada lo detectara, porque las pruebas de esos controladores usan `MockMvcBuilders.standaloneSetup()`, que no levanta la cadena de filtros de Spring Security: verifican el mapeo HTTP, no la autorización.

La corrección no aplica una regla uniforme, sino una por recurso según el dato que maneja:

Recurso	Acceso	Criterio
Persona	Solo ADMINISTRADOR	Concentra la identificación de res; ningún otro rol necesita op
Usuario	Solo ADMINISTRADOR	Ya lo estaba, a nivel de clase.
Categoria	Lectura: 3 roles Escritura: ADMINISTRADOR	El formulario de estudiante c las categorías activas; escribir al catálogo del que dependen por c ránea.
Entrenador	Lectura: ADMIN, ENTRENADOR Escritura: ADMINISTRADOR	El alta y la baja crean o ret vínculo con una cuenta.
EstadoGeneral	Lectura: 3 roles	Catálogo sin datos personales.

La evidencia regenerada (`docs/mediciones/sec/a01-acceso-roto.txt`) comprueba las dos direcciones, que es lo que hace verificable la corrección: 403 en las siete operaciones administrativas —incluida la búsqueda por cédula— y 200 en las dos lecturas que un rol USER sí debe poder hacer. Sin esa segunda mitad, la evidencia sería compatible con haber roto la aplicación en vez de haberla asegurado.

8.3. Dos defectos de funcionamiento encontrados al ejecutar el sistema

Ninguno de los dos aparece si el sistema no se levanta y se usa. Se documentan porque ilustran una limitación concreta de la estrategia de pruebas de este proyecto, no solo porque estaban.

El registro de usuarios estaba completamente roto (RF-01). `POST /api/auth/registro` devolvía 500 ante *cualquier* petición. La reestructuración convirtió `cedula`, `correo` y `fecha_nacimiento` en columnas NOT NULL de `seguridad.personas`, pero `RegisterRequest` nunca las pidió; el alta violaba la restricción de integridad en cada intento. Tampoco se asignaba `id_estado_general`, igualmente obligatorio.

Lo relevante para la evaluación es *por qué no lo detectaron las pruebas*. `AuthServiceTest` cubre el registro con dos casos y ambos pasaban: los repositorios están mockeados, de modo que `personaRepository.save()` devuelve el objeto que se le pasa sin consultar ninguna base de datos. La restricción `NOT NULL` la aplica PostgreSQL, y en una prueba con dobles PostgreSQL no participa. Es el límite natural de la prueba unitaria con mocks: verifica la lógica que escribimos, no el contrato con el motor de base de datos. La prueba que se agregó no finge resolver eso —haría falta una prueba de integración con base real—, sino que cubre el flanco que sí es verificable sin ella: que una petición sin esos campos no supere la validación y por tanto nunca llegue a la base.

Un cuerpo mal formado se reportaba como fallo del servidor. `HttpMessageNotReadableException` no tenía manejador en `GlobalExceptionHandler` y caía en el genérico, devolviendo 500 «Error interno del servidor» ante un error que es del cliente. Además de ser semánticamente incorrecto según RFC 9457 [9], ensuciaba el registro con trazas de pila que no corresponden a ningún defecto del sistema —ruido que compite con las anomalías reales en A09. Ahora responde 400.

8.4. Cobertura de pruebas

Medido el 2026-07-30 con construcción limpia (`./mvnw clean test`): 72,5 % de cobertura de instrucciones (2507 cubiertas de 3457), con 102 pruebas y ningún fallo. Cumple el umbral del 60 %. El camino hasta este número quedó documentado en vez de solo reportar el resultado final: al medir por primera vez después de la reestructuración de paquetes (sección 3) el resultado real era **39,8 %** — los recursos nuevos (`Categoria`, `Entrenador`, `Usuario`, `Persona`, `EstadoGeneral`) no tenían ninguna prueba propia. Se agregaron 57 pruebas nuevas para esos cinco recursos (10 clases: servicio y controlador de cada uno), y la cobertura subió por encima del 70 %.

La palabra «limpia» está ahí por una razón concreta. La primera medición posterior a la reestructuración se hizo con `./mvnw test` sobre un directorio `target/` que todavía conservaba archivos `.class` compilados *antes* del cambio de paquetes. El reporte que se llegó a archivar como evidencia listaba paquetes que ya no existen en el código fuente (`org.uteq.backend.auth.*`, `org.uteq.backend.estudiante.*`): JaCoCo instrumenta lo que encuentra en `target/classes`, no lo que hay en `src/`. La cifra publicada aquí proviene de `clean test` y el reporte archivado en `docs/mediciones/jacoco/` contiene exactamente los 27 paquetes reales. La diferencia numérica entre ambas mediciones era de apenas unas décimas, y ese es justamente el punto: una evidencia de cobertura que incluye código inexistente no es evidencia válida *aunque* el porcentaje salga parecido, porque nada garantiza que la próxima vez la distorsión sea igual de pequeña. Para que el defecto no pueda repetirse, el objetivo `make test` pasó a ejecutar `clean test`.

Clase de prueba	Nº	Qué verifica
<code>AuthServiceTest</code>	6	Login correcto e incorrecto, registro, registro duplicado, registro incompleto rechazado con 422, disponibilidad.
<code>JwtServiceTest</code>	6	Emisor y audiencia válidos, audiencia distinta rechazada, token manipulado, refresco.
<code>LoginAttemptServiceTest</code>	6	Bloqueo al alcanzar el límite, no reinicio del TTL, borrado del contador al acertar.
<code>RedisBlacklistServiceTest</code>	6	Revocación con TTL restante, TTL no positivo ignorado, consulta de revocación.
<code>EstudianteControllerTest</code>	9	Los siete verbos del recurso más 404 y 422.
<code>EstudianteServiceTest</code>	11	Paginación envuelta, baja lógica, persistencia transaccional, delegación al procedimiento SQL.
<code>BackendApplicationTests</code>	1	Arranque del contexto contra el esquema migrado.
<code>CategoriaServiceTest</code>	9	Paginación, alta, edición, baja lógica, validación de rango de edad.
<code>CategoriaControllerTest</code>	7	Mapeo HTTP: 200/201/204/400/404/422.
<code>EntrenadorServiceTest</code>	6	Paginación con mapeo persona/usuario, persona o usuario duplicados, alta, baja lógica.
<code>EntrenadorControllerTest</code>	5	Mapeo HTTP: 200/201/204/404/422.
<code>UsuarioServiceTest</code>	6	Paginación, username duplicado, alta con contraseña codificada, persona inexistente, baja lógica.
<code>UsuarioControllerTest</code>	6	Mapeo HTTP: 200/201/204/400/422.
<code>PersonaServiceTest</code>	8	Paginación, búsqueda por cédula, unicidad de cédula/correo al crear y editar, baja lógica.
<code>PersonaControllerTest</code>	6	Mapeo HTTP: 200/201/204/400/422.
<code>EstadoGeneralServiceTest</code>	2	Listado completo y listado vacío.
<code>EstadoGeneralControllerTest</code>	1	Mapeo HTTP del único <i>endpoint</i> .
Total	102	

Un detalle técnico real que surgió al escribir estas pruebas: los cuatro controladores que reciben `Pageable` como parámetro (`Categoria`, `Entrenador`, `Usuario`, `Persona`) fallaban con 500 bajo `MockMvcBuilders.standaloneSetup()`, porque ese modo de arranque no registra el `PageableHandlerMethodArgumentResolver` que sí provee el contexto completo de Spring — `EstudianteController` no lo sufre porque usa `@RequestParam` planos en vez de `Pageable`. Se resolvió registrando el resolutor explícitamente en cada prueba.

Un detalle relevante de una entrega anterior: se había descubierto que JaCoCo *nunca* había generado cobertura real, porque el `argLine` configurado en Surefire sobrescribía el *javaagent* del propio JaCoCo. Ese defecto está corregido; por eso esta medición pudo mostrar honestamente primero una regresión y después la mejora real.

Las pruebas de `LoginAttemptServiceTest` merecen mención aparte: verifican no solo que el bloqueo ocurra, sino que un fallo posterior *no* reinicie la ventana de tiempo. Sin esa segunda propiedad, un atacante podría mantener la cuenta en estado de bloqueo indefinido, o bien —según cómo se implemente— reiniciar el contador y evadir el límite.

8.5. Calidad web: Lighthouse (Bloque C.5)

Tres corridas con perfil móvil (Lighthouse v13.4.1):

Categoría	Media	Umbral	Estado
Rendimiento	92,3	≥ 80	OK
Accesibilidad	100	≥ 90	OK
Buenas prácticas	96	≥ 90	OK
SEO	63	≥ 90	Ver §8.5.1

Métricas Web Vitals: LCP 2,86 s; FCP 2,43 s; TBT 25,7 ms; y CLS **0,000** en las tres corridas (la interfaz no desplaza contenido durante la carga).

Esta medición expuso, y permitió corregir, tres defectos reales del frontend: el documento declaraba `lang="en"` en una aplicación íntegramente en español; conservaba el título **Frontend** generado por Angular CLI y no tenía descripción; y carecía de un elemento `<main>`, lo que hacía fallar la auditoría de puntos de referencia de accesibilidad. Corregidos los tres, la accesibilidad pasó de 97 a **100**.

Se corrigió además un defecto en la propia configuración de medición: `lighthouse.js` declaraba `preset: 'mobile'`, un valor que no existe en Lighthouse, lo que abortaba toda corrida con código de salida 1. Es decir, **esta medición nunca se había podido ejecutar**.

8.5.1. Por qué el SEO de 63 es el resultado correcto

El puntaje SEO bajó de 82 a 63 *a causa de una corrección deliberada*. Al añadir un `robots.txt` válido con `Disallow: /`, Lighthouse activa la penalización `is-crawlable` («página bloqueada para indexación»), que por sí sola descuenta 27 puntos, porque su categoría SEO asume que el sitio *quiere* ser indexado.

SGED es una aplicación de gestión interna que trata datos personales de menores de edad. Que fuese indexable por buscadores sería un defecto de privacidad, no una virtud. Elevar el SEO a 90 exigiría eliminar el `robots.txt`: degradar la privacidad del sistema para mejorar una métrica. Se optó por lo contrario, se relajó el umbral agregado de esa categoría a advertencia con la justificación escrita en el propio archivo de configuración, y se mantuvieron como bloqueantes las auditorías SEO que sí aplican a una aplicación interna (`meta-description`, `document-title`, `html-has-lang`, `viewport`), todas ellas en 1,0. **OK**

8.6. Usabilidad: SUS (Bloque C.3)

El instrumento (`docs/mediciones/sus/INSTRUMENTO-SUS.md`) es el cuestionario SUS de diez ítems [4] con la escala adjetival de interpretación [3]. El script `scripts/sus-analysis.py` calcula media, desviación típica, intervalo de confianza al 95 %, mediana y grado por participante; se niega deliberadamente a ejecutarse con menos de diez participantes, y su aritmética quedó verificada contra los cuatro casos de referencia de la literatura (0, 50, 75 y 100 puntos) antes de usarlo con datos reales.

Aplicado el 2026-07-30 a 10 participantes (formularios en papel, transcritos a `docs/mediciones/sus/respuestas` sin registrar nombres, solo un código de participante y su perfil): 3 entrenadores, 3 recepcionistas, 2 estudiantes, 2 padres de familia.

Métrica	Valor
Media SUS	68,25
Desviación típica	22,14
IC 95 %	68,25 $\pm$ 13,72 (54,53 – 81,97)
Mediana	73,75
Grado	C — Aceptable

La media cruza el umbral de 68 por apenas 0,25 puntos, y el intervalo de confianza es ancho: con esta muestra no puede afirmarse con confianza estadística que la media poblacional real supere 68, solo que 68,25 es la mejor estimación puntual disponible. No se presenta como un resultado cómodo.

El hallazgo más informativo no es la media, es su distribución bimodal por perfil, que la agregación esconde:

Perfil	Promedio SUS	Grado
Entrenador (n=3)	86,7	A
Estudiante (n=2)	90,0	A
Recepcionista (n=3)	51,7	F
Padre de familia (n=2)	43,75	F

Entrenadores y estudiantes calificaron el sistema como excelente; recepcionistas y padres de familia, como inaceptable. El CRUD de estudiantes implementado hasta esta entrega (RF-08 a RF-15) es exactamente el flujo operado por un rol administrativo — no el de un entrenador consultando información, ni el de un padre buscando visibilidad sobre su representado —, lo que sugiere que la interfaz actual está más pulida para un caso de uso que no es el de quienes peor la calificaron. Es una pista concreta de dónde enfocar el trabajo de usabilidad antes de la Entrega Final, no solo un número que aprobó por poco.

9. Reproducibilidad

El sistema se levanta completo desde una clonación limpia con un solo comando:

```
1 git clone https://github.com/DarwinSM21/SGED_APPWEB.git
2 cd SGED_APPWEB
3 cp .env.example .env
4 make up
```

Listing 4: Arranque reproducible

Objetivo	Acción
<code>make up</code>	Levanta el sistema completo
<code>make test</code>	JUnit 5 + cobertura JaCoCo
<code>make bench</code>	3 corridas k6 + análisis con IC 95 %
<code>make audit</code>	Auditoría OWASP (6 controles) + SQL dinámico
<code>make clean</code>	Limpia contenedores, volúmenes y compilaciones

Las imágenes de PostgreSQL y Redis están fijadas por digest SHA-256, no por etiqueta móvil: dos construcciones del mismo *commit* usan exactamente los mismos binarios. La reproducibilidad se verificó clonando el repositorio en un directorio independiente, no solo reiniciando el volumen local.

10. Amenazas a la validez

Los resultados de la sección anterior cumplen los umbrales exigidos, pero cumplirlos no equivale a ser generalizables. Esta sección declara las limitaciones que acotan hasta dónde puede sostenerse cada conclusión, siguiendo la clasificación habitual en ingeniería de software empírica [6].

10.1. Validez de constructo

¿Mide cada indicador lo que se pretende medir?

- **La cobertura de instrucciones no mide calidad de las pruebas.** Un 72,5 % indica qué proporción del código se ejecuta durante la suite, no que las aserciones sean pertinentes. El caso del registro de usuarios lo demuestra dentro de este mismo proyecto: el *endpoint* estaba cubierto por dos pruebas que pasaban y aun así fallaba en toda petición real (sección 8.3).
- **La auditoría OWASP verifica controles, no ausencia de vulnerabilidades.** Comprueba seis controles concretos del Top 10 con peticiones dirigidas. No es una prueba de penetración ni un análisis estático: que los seis den correcto no permite afirmar que el sistema sea seguro, solo que esos seis comportamientos son los esperados.
- **El SUS mide usabilidad percibida, no desempeño.** No se registraron tiempos ni tasas de finalización por tarea, de modo que un participante pudo puntuar alto sin haber completado las tareas propuestas. Las columnas de completitud del CSV quedaron vacías y se declaran vacías en lugar de rellenarse.

10.2. Validez interna

¿Puede algo distinto de lo declarado explicar los resultados?

- **Las mediciones de rendimiento se tomaron en la misma máquina que ejecuta el sistema,** sin latencia de red real. Los 14,18 ms de p95 son un límite inferior optimista: en un despliegue con red de por medio la cifra sería mayor, aunque el margen frente al umbral de 200 ms es amplio.
- **Tres corridas son pocas** para caracterizar una distribución. Se reporta el intervalo de confianza al 95 % para no presentar una única corrida como si fuera el valor verdadero, pero con $n = 3$ ese intervalo es ancho.
- **La base de datos de las mediciones contiene datos sembrados, no un volumen de producción.** El rendimiento de las consultas paginadas sobre un catálogo pequeño no anticipa el comportamiento con miles de estudiantes.

10.3. Validez externa

¿Hasta dónde se generalizan los resultados?

- **La muestra del SUS es de 10 participantes,** el mínimo metodológico, y por conveniencia. El intervalo de confianza (54,53–81,97) es lo bastante ancho como para que no pueda afirmarse con confianza estadística que la media poblacional supere el umbral de 68: 68,25 es la mejor estimación puntual disponible, no un aprobado consolidado.
- **Los subgrupos por perfil tienen 2 o 3 personas cada uno.** El patrón bimodal es nítido y tiene una explicación plausible ligada a qué flujos están implementados, pero con esos tamaños no puede descartarse que parte de la diferencia sea variación individual. Se presenta como hipótesis orientadora para la Entrega Final, no como conclusión establecida.
- **Los resultados describen este sistema en este entorno.** No se comparan contra un sistema de referencia ni contra una versión anterior medida en las mismas condiciones.

10.4. Validez de conclusión

El riesgo principal de este informe no es estadístico sino de procedimiento, y ya se materializó tres veces: **un indicador puede dar verde sobre un sistema que no hace lo que el indicador afirma** (sección 8.2.1). Las tres ocurrencias se corrigieron en su origen, pero la lección se declara aquí porque acota la confianza que merece cualquier cifra de este documento: son válidas en tanto la medición se ejecute contra el sistema real, desde estado limpio, y verificando las dos direcciones del comportamiento esperado.

Queda además una divergencia sin resolver que afecta la reproducibilidad a futuro: el esquema consolidado que crea la base de datos (`db/schema.sql`) y las migraciones Flyway versionadas describen estructuras distintas, porque las segundas no se actualizaron tras la reestructuración y hoy no se ejecutan (`FLYWAY_ENABLED=false`). El sistema arranca de forma reproducible por la primera vía, que es la verificada en este informe, pero la decisión sobre cuál de las dos fuentes queda como autoritativa está pendiente y se declara abierta.

Finalmente, **el DOI de Zenodo no está asignado** a la etiqueta `v0.9.0-rc`. Sin él, el archivo permanente exigido por el Bloque E.2 no está satisfecho y la portada de este informe no puede declararlo. Se consigna como incumplimiento, no como omisión.

11. Consideraciones éticas

El sistema trata datos personales de niños, niñas y adolescentes: nombres, cédula, fecha de nacimiento, asistencia con hora y lugar, y evaluaciones individuales de desempeño. Esa combinación es sensible porque los titulares son menores que no pueden consentir por sí mismos, porque la asistencia es información de localización temporal, y porque las evaluaciones constituyen perfilado de personas [1, 2].

El principio de minimización se aplicó al diseño original —se descartaron contextura y datos de alimentación—, pero conviene ser preciso sobre su alcance real, porque la versión anterior de este informe afirmaba que tampoco se recolectaban peso y altura y **esa afirmación ya no es cierta**: la reestructuración incorporó ambas columnas a `academico.estudiantes` y son exponibles por la API. El documento de ética se corrigió (hallazgo H-06) en lugar de mantener la declaración cómoda, y este informe se corrige aquí por la misma razón: una declaración de minimización que no coincide con el esquema no protege a nadie.

`docs/etica/ETHICS.md` documenta ocho hallazgos, en lugar de afirmar que todo está resuelto: la cédula se almacena sin validación ni cifrado (H-01); las observaciones del entrenador son texto libre sin control sobre un menor (H-02); no existe mecanismo de supresión de datos —la baja lógica preserva el historial pero no es un borrado— (H-03); el consentimiento del representante no está modelado, lo que bloquea el módulo de notificaciones (H-04); el certificado TLS es autofirmado, adecuado para evaluación pero no para producción (H-05); peso y altura se agregaron sin base legal documentada (H-06); y la plantilla de consentimiento cubre a los evaluadores del SUS, no a los representantes de los menores (H-07).

El octavo, **H-08**, se registra ya corregido, y se deja escrito justamente porque corregirlo en silencio habría sido lo fácil: los datos identificativos de los estudiantes estuvieron accesibles a cualquier cuenta autenticada del sistema, incluida la búsqueda por número de cédula (sección 8.2.1). Un documento de ética que solo enumera riesgos hipotéticos y omite la exposición concreta que sí ocurrió sería un documento de imagen, no de rendición de cuentas.

Todos los datos del repositorio son ficticios. No se utilizaron datos reales de menores en ninguna etapa del desarrollo ni de las mediciones.

12. Declaración de contribuciones (CRedit)

Los roles siguientes no son autodeclarados: se derivan del historial de `git` del repositorio y cada uno puede verificarse con `git log -author`. Las cifras provienen de `git log --numstat` sobre la rama `main` y separan lo *escrito* de lo *generado* (reportes de JaCoCo y Lighthouse, salidas crudas de k6, `package-lock.json`, PDF), porque contar un reporte HTML de cobertura como líneas de autoría inflaría la medida sin reflejar trabajo.

Integrante	Commits	Líneas escritas	Archivos
Pallo Pinto Alejandro Daniel	72	17 760	374
Arcalle Grefa Darwin Orlando	17	5 429	303
Velez Lopez Ricardo Elias	32	4 077	108
Total en main	121	27 266	—

12.1. Roles por integrante

Pallo Pinto Alejandro Daniel — *Software, Validación, Curación de datos, Redacción (borrador original), Visualización.*

Responsable de la totalidad del eje de verificación que define esta entrega. En seguridad: migración del JWT de `localStorage` a cookie `HttpOnly+Secure+SameSite`, terminación TLS, CSP explícito, registro de auditoría A09, y la corrección del control de acceso ausente en los cinco recursos nuevos (H-08, sección 8.2.1). En datos: conversión de las funciones PL/pgSQL a procedimientos reales invocables con `@Procedure` (sección 5) y fijación de imágenes por digest. En validación: las 102 pruebas actuales —incluidas las 57 que cerraron la regresión de cobertura—, la corrección del defecto por el que JaCoCo nunca había medido cobertura real, y las tres campañas de medición empírica (k6, OWASP, Lighthouse) junto con sus *scripts* de análisis. En usabilidad: instrumento SUS, *script* de cálculo validado contra los casos de referencia de la literatura, y transcripción de las diez encuestas. En redacción: este informe y el corpus documental (SRS, casos de uso, historias, matriz de trazabilidad, ADR, ética, diccionario de datos, gobernanza del repositorio).

Arcalle Grefa Darwin Orlando — *Conceptualización, Software, Administración del proyecto.*

Estructura inicial del repositorio y del modelo de datos (`DataBase/`), trabajo sobre el *frontend* Angular (componentes, *Dockerfile* y configuración de formato), y administración del repositorio remoto, del que es propietario. Participación en la reestructuración en dominios `academico/deportivo/seguridad` descrita en la sección 3.

Velez Lopez Ricardo Elias — *Conceptualización, Software, Metodología.*

Arranque del *frontend* Angular y del módulo de autenticación, CRUD inicial de **Estudiante**, y aportes a `docs/requisitos/` y `docs/observaciones/`. Autor principal de la reestructuración en tres dominios y de los cinco recursos CRUD nuevos, que es el cambio estructural sobre el que se apoya esta entrega.

12.2. Dos advertencias sobre la medida

El volumen no es la contribución. Las líneas escritas miden actividad, no valor: un cambio de dos líneas que corrige un fallo de control de acceso pesa más que dos mil líneas de documentación. La tabla se incluye porque el criterio de evaluación exige autoría verificable, no para establecer una jerarquía entre integrantes.

Dos commits figuran bajo la cuenta DannaN24 (`dninasuntar@uteq.edu.ec`), del 2026-06-29, y son trabajo de **Pallo Pinto Alejandro Daniel**: esa era la sesión configurada en el equipo de la universidad que utilizó ese día. Ya quedaba insinuado en el propio historial (*commit*

a456255, del mismo día). Están sumados a su fila en la tabla anterior. No se reescribió el historial de `git` —hacerlo sobre ramas ya compartidas con el equipo habría roto sus copias locales—; la aclaración se hace por escrito aquí y en `CONTRIBUTORS.md`, que es el mecanismo apropiado para una corrección de autoría a posteriori.

Adicionalmente, varios `commits` de Ricardo y Darwin usan correos personales en vez del institucional, y la cuenta de Ricardo aparece con dos identidades (`ricardoforever@outlook.es` y `ricardo@email.com`). Las cifras de la tabla las consolidan como una sola persona. Se recomienda unificar `git config user.email` antes de la Entrega Final.

13. Trazabilidad y honestidad académica

La matriz `docs/trazabilidad/matriz.csv` enlaza cada requisito con su historia de usuario, su caso de uso, su `endpoint`, el archivo que lo implementa, la prueba JUnit que lo cubre y la evidencia empírica correspondiente.

El historial de `git` evidencia la autoría de cada integrante. Se documenta en `CONTRIBUTORS.md` que dos `commits` figuran bajo una cuenta distinta por haber sido realizados desde un equipo de la universidad con otra sesión configurada; la aclaración se hace por escrito en lugar de reescribir el historial ya compartido. El mismo archivo declara explícitamente el uso de herramientas de asistencia por inteligencia artificial, siguiendo la guía de transparencia del SWEBOK v4.0 [14].

14. Conclusiones

1. El sistema cumple con holgura los umbrales cuantitativos exigidos: p95 de 14,18 ms frente a un límite de 200 ms, 6 de 6 controles OWASP verificados con evidencia reproducible, accesibilidad Lighthouse de 100/100, y **72,5 %** de cobertura de pruebas frente a un mínimo de 60 % (sección 8.4) — cifra alcanzada después de detectar y corregir una regresión real a 39,8 % causada por los recursos nuevos de la reestructuración, no reportando directamente un número cómodo.
2. **Una medición que no se ejecuta contra el sistema real no es evidencia.** Es la lección transversal de esta entrega, y aparece tres veces con la misma forma. La cobertura de JaCoCo se midió sobre un `target/` con clases previas a la reestructuración e incluía paquetes que ya no existen. La auditoría OWASP A01 devolvía 401 en todo y eso se parecía a un resultado correcto, cuando en realidad su propio control A07 la había dejado bloqueada. Y el registro de usuarios (RF-01) estaba roto contra la base de datos desde la reestructuración mientras sus dos pruebas unitarias pasaban, porque mockean el repositorio y la restricción la aplica PostgreSQL. En los tres casos el indicador era verde y el sistema no hacía lo que el indicador decía. Las tres mediciones se corrigieron en su origen — `make test` pasa a `clean test`, la auditoría limpia su propio contador y comprueba recurso por recurso— para que el defecto no pueda repetirse sin ser visible.
3. El aporte más sustantivo de esta entrega no fue añadir funcionalidad, sino *verificar* lo que ya existía, dos veces: una vez contra el código de la entrega original, y una segunda vez después de que otros dos integrantes reestructuraran el backend en paralelo. Ambas rondas revelaron defectos que ninguna revisión superficial había detectado — desde que JaCoCo no medía cobertura en absoluto, hasta que un ADR de arquitectura describía un mecanismo de autenticación (JWT en `localStorage`) distinto del que el código realmente usa (cookie `HttpOnly`), pasando por el control de acceso ausente en los cinco recursos nuevos (H-08). Todos los defectos encontrados se corrigieron y quedan documentados en las secciones 3 y 8.3 en vez de ocultarse.

4. La distinción entre lo implementado y lo modelado se mantiene explícita en todo el documento. En el caso de **Representante** y **Equipo** se llevó un paso más allá: en vez de dejar catorce clases vacías —una de ellas de cero bytes, el mismo defecto que motivó tres observaciones del docente en la Entrega 1B— se eliminaron del código y ambos módulos quedan declarados como pendientes solo en la documentación (sección 3). Un archivo vacío no cuenta como funcionalidad entregada, y tampoco debería quedarse en el repositorio aparentando que sí.
5. La encuesta SUS (sección 8.6) ya no es un frente abierto, pero su resultado abre uno nuevo: el sistema es excelente para entrenadores y estudiantes (grado A) e inaceptable para receptionistas y padres de familia (grado F). Mejorar la usabilidad del flujo administrativo y del futuro módulo de representantes es, con evidencia real detrás, más urgente que subir la media agregada. La exposición REST del dominio deportivo restante (horarios, sesiones, asistencias, evaluaciones) sigue siendo el objetivo natural de la Entrega Final.

Referencias

- [1] Asamblea Nacional del Ecuador. Ley orgánica de protección de datos personales. Registro Oficial Suplemento 459, 26 de mayo de 2021, 2021.
- [2] Association for Computing Machinery. ACM code of ethics and professional conduct. <https://www.acm.org/code-of-ethics>, 2018.
- [3] Aaron Bangor, Philip Kortum, and James Miller. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies*, 4(3):114–123, 2009.
- [4] John Brooke. SUS: A ‘quick and dirty’ usability scale. In *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.
- [5] Simon Brown. *Software Architecture for Developers, Volume 2: Visualise, document and explore your software architecture*. Leanpub, 2018. Modelo C4.
- [6] ISO/IEC. ISO/IEC 25010:2011 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models, 2011.
- [7] ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018 — systems and software engineering — life cycle processes — requirements engineering, 2018.
- [8] M. B. Jones, J. Bradley, and N. Sakimura. JSON web token (JWT). Request for Comments 7519, Internet Engineering Task Force (IETF), May 2015. <https://www.rfc-editor.org/rfc/rfc7519>.
- [9] M. Nottingham, E. Wilde, and S. Dalal. Problem details for HTTP APIs. Request for Comments 9457, Internet Engineering Task Force (IETF), July 2023. <https://www.rfc-editor.org/rfc/rfc9457>.
- [10] Michael T. Nygard. Documenting architecture decisions. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>, 2011.
- [11] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks. <https://owasp.org/Top10/>, 2021.
- [12] PostgreSQL Global Development Group. PostgreSQL 16 documentation — CREATE PROCEDURE. <https://www.postgresql.org/docs/16/sql-createprocedure.html>, 2026.
- [13] Redgate. Flyway documentation — migrations. <https://documentation.red-gate.com/flyway/>, 2026.
- [14] Hironori Washizaki, editor. *SWEBOK Guide v4.0: Guide to the Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.